



Sistema de Gestão de Biblioteca

Lucas Fernandes Candido da Silva - Aspirante

Matheus Emanuel de Souza Agra - Aspirante

Maysa Barros Campelo - Aspirante

Safira Moraes Gomes - Aspirante

Luiz Philip Santiago da Silva - Gerente

SUMÁRIO

O que é?	3
Interface Web, voltada para administradores:	3
Interface Mobile, voltada para leitores:	3
Backend e infraestrutura do sistema:	3
Qual é o intuito?	4
Como utilizar?	5
Para Administradores (Web):	5
Para Leitores (Mobile):	6
Como manter?	7
Conservação no banco de dados:	7
Manutenção da tabela do histórico de empréstimos:	7
Verificação de Consistência do Estoque:	7
Monitoramento de Erros em Tempo Real:	8
Atualização tecnológica e gestão de dependências:	8
Como escalar a solução?	9
Otimização nas consultas:	9
Implementação de autenticação:	9
Ampliação dos Canais de Comunicação:	10
Automação de Implantação e Testes:	10
Informações ao usuário	11
Tecnologias utilizadas	12
Arquitetura do projeto	16
Frontend (client):	16
Backend (server):	16
mobile:	17
Estrutura e organização do projeto	18
Links relevantes	29
Comentários	32
Lucas Fernandes:	32
Matheus Agra:	33
Maysa Barros:	34
Safira Moraes:	35
Nota final	37

O que é?

A solução desenvolvida é um Sistema de Gestão de Biblioteca, criado para facilitar a organização e o acompanhamento dos livros e empréstimos de uma biblioteca escolar. O sistema funciona de forma integrada entre a versão web e a versão mobile, permitindo que administradores e usuários tenham acesso às informações da biblioteca de maneira simples e prática. A proposta do sistema é tornar o processo de controle da biblioteca mais ágil, reduzindo a dependência de registros manuais e facilitando a consulta de informações importantes, como livros cadastrados, empréstimos ativos, datas de devolução e possíveis atrasos. A solução é composta por três partes principais:

Interface Web, voltada para administradores:

A versão web é destinada aos administradores da biblioteca. Por meio dela, é possível acompanhar informações gerais do sistema, visualizar livros cadastrados, registrar novos livros e cadastrar empréstimos.

Contando também com uma tela de dashboard, que apresenta métricas importantes da biblioteca, como a quantidade de livros, empréstimos ativos e livros em atraso. Além disso, o sistema pode exibir gráficos que ajudam o administrador a compreender melhor a distribuição dos livros por categoria.

Interface Mobile, voltada para leitores:

A versão mobile é destinada aos usuários que desejam consultar informações da biblioteca pelo celular. Pelo aplicativo, é possível buscar por livros ou por nome de usuário para visualizar os empréstimos associados. Ao realizar uma consulta, o usuário consegue visualizar informações como o livro relacionado ao empréstimo, o nome do usuário, a capa do livro, a data de empréstimo e a data de devolução. Dessa forma, o aplicativo torna o acesso às informações mais rápido e direto.

Backend e infraestrutura do sistema:

Além das interfaces web e mobile, a solução conta com uma estrutura responsável por armazenar, processar e organizar os dados do sistema. Essa estrutura permite que as informações cadastradas sejam mantidas e utilizadas pelas duas versões da aplicação.

Sendo também responsável pelas regras de negócio que garantem o funcionamento do sistema, como o controle de empréstimos, a identificação de atrasos e a comunicação entre as telas e os dados cadastrados. Além disso, o sistema possui integração para envio de e-mails automáticos, que podem ser utilizados para lembrar usuários sobre atrasos na devolução de livros.

Qual é o intuito?

O intuito da aplicação é melhorar a rotina de controle de uma biblioteca escolar, tornando o processo de acompanhamento dos empréstimos mais confiável, acessível e menos sujeito a falhas. A solução busca atender a uma necessidade comum em bibliotecas: manter informações importantes organizadas e disponíveis de forma rápida para quem administra o acervo e para quem consulta os registros de empréstimos.

A aplicação também tem como propósito reduzir dificuldades causadas por controles descentralizados, como anotações manuais, planilhas separadas ou registros que podem se perder com facilidade. Ao concentrar essas informações em um sistema, a biblioteca passa a ter mais segurança no acompanhamento dos livros, dos prazos e das movimentações realizadas.

Outro objetivo importante é facilitar a identificação de pendências, como empréstimos em atraso, permitindo que a administração tenha uma visão mais clara da situação da biblioteca e possa agir de forma mais rápida quando necessário.

Para os usuários, o intuito é tornar o acesso às informações mais simples e transparente, permitindo que consultem os registros disponíveis sem depender exclusivamente do contato direto com a administração da biblioteca.

Como utilizar?

Nesta seção, são descritas as principais ações que podem ser realizadas nas versões web e mobile do sistema.

Para Administradores (Web):

- Para acompanhar as métricas da biblioteca, o administrador deve acessar a tela de Dashboard. Nessa tela, são exibidos indicadores como livros cadastrados, empréstimos ativos e livros em atraso. Além dos indicadores gerais, o Dashboard apresenta três tipos de gráficos: quantidade de livros por categoria, quantidade de empréstimos por categoria e livros com maior quantidade de empréstimos ativos. A tela também exibe os cinco últimos empréstimos realizados.
- Para visualizar os livros cadastrados, o administrador deve acessar a tela de Livros. Nessa tela, é possível consultar os livros disponíveis no sistema e visualizar suas principais informações.
- Para cadastrar um empréstimo, o administrador deve acessar a tela de Livros, selecionar o livro desejado e clicar na opção de “Emprestar”. Em seguida, deve preencher as informações necessárias: nome do usuário, e-mail, data de locação e data de devolução. Após o cadastro, o empréstimo fica registrado no sistema.
- Para visualizar os detalhes de um livro, o administrador deve acessar a tela de Livros, selecionar o livro desejado e clicar na opção “Ver”. Nessa área, são exibidas informações detalhadas sobre o livro e os empréstimos associados a ele. Para confirmar uma devolução, o administrador deve acessar a página de detalhes do livro pela opção “Ver”. Nessa área, é possível localizar o empréstimo correspondente e confirmar que o livro foi devolvido.
- Para enviar um e-mail informando atraso, o administrador também deve acessar a página de detalhes do livro pela opção “Ver”. Nessa área, é possível identificar empréstimos em atraso e enviar uma notificação ao usuário responsável.
- Para cadastrar um novo livro, o administrador deve acessar a tela de Novo Livro. Nessa tela, deve preencher os dados necessários: nome do livro, autor, ISBN, editora, ano de lançamento, quantidade disponível e categoria. Após o cadastro, o livro passa a aparecer na lista de livros do sistema.

Para Leitores (Mobile):

- Para visualizar os empréstimos, o usuário deve acessar a tela principal do aplicativo. Nessa tela, são exibidas informações como o nome do usuário, o livro relacionado, a capa do livro, a data de empréstimo e a data de devolução, quando esses dados estiverem disponíveis.
- Para consultar os empréstimos associados a um livro específico, o usuário deve utilizar o campo de busca e pesquisar pelo nome do livro. Após a pesquisa, o aplicativo exibe todos os empréstimos relacionados àquele livro, permitindo acompanhar quais registros estão associados a ele.
- Para consultar os empréstimos associados a um usuário específico, o usuário deve utilizar o campo de busca e pesquisar pelo nome do usuário. Com isso, o aplicativo apresenta todos os empréstimos vinculados àquele usuário, facilitando a visualização das informações de forma mais direta.

Como manter?

A manutenção sustentável, a longo prazo, do ecossistema depende da execução de diretrizes que preservem a segurança dos dados e a alta disponibilidade dos serviços. Com o objetivo de assegurar a estabilidade operacional, a integridade das informações e a fluidez das interfaces sob cenários de alta concorrência, estabelecem-se pontos, tais quais os apresentados abaixo:

Conservação no banco de dados:

Para evitar riscos de perda de informações ou corrupção de arquivos, deve-se estabelecer uma rotina de *backups* diários automatizados. Essas cópias de segurança precisam ser armazenadas em servidores externos isolados (nuvem), os quais garantem a recuperação rápida do estado do sistema em caso de problemas ou falhas de infraestrutura. O desenvolvedor encarregado de validar a integridade dessas rotinas ou restaurar um estado anterior do banco de dados deve inspecionar as configurações de persistência estruturadas na pasta **server/src/database/** e, para testar a sincronização local e aplicar modificações estruturais controladas, utilizar o utilitário do ORM por meio do comando **cd server && npx prisma db push**.

Manutenção da tabela do histórico de empréstimos:

O crescimento volumétrico da tabela de empréstimos pode degradar o desempenho das consultas operacionais. Para solucionar este problema, adota-se a estratégia de arquivamento de dados, movendo os registros sinalizados com o status "Devolvido" para uma tabela histórica de auditoria, mantendo a tabela principal enxuta e otimizada apenas para transações ativas. O mapeamento dessas entidades e seus respectivos relacionamentos deve ser gerenciado e verificado no arquivo central de modelagem em **server/prisma/schema.prisma**. Caso novas colunas sejam integradas a essa estrutura de auditoria, o desenvolvedor deverá gerar e aplicar a respectiva migração ao ambiente executando o comando **cd server && npx prisma migrate dev**.

Verificação de Consistência do Estoque:

Implementação de rotinas periódicas de conciliação para mitigar riscos de desvios no inventário automatizado. O sistema executa processos computacionais que confrontam o saldo do campo de quantidade disponível de cada livro com o volume de empréstimos ativos registrados. Essa lógica de validação das regras de negócio do acervo fica centralizada no arquivo **server/src/services/loan/loanService.ts**, local onde o desenvolvedor deve

inspecionar o código caso necessite depurar o fluxo de entrada e saída de títulos. Quaisquer anomalias detectadas disparam alertas automáticos aos administradores, e o comportamento dessa estrutura pode ser validado localmente executando a suíte de testes com o comando **cd server && pnpm test**.

Monitoramento de Erros em Tempo Real:

. O objetivo é permitir que a equipe técnica atue preventivamente na correção de bugs, assegurando a continuidade do serviço.

Implementação de ferramentas de rastreamento para identificar falhas no sistema antes que elas afetem a experiência do usuário. O sistema deve registrar *logs* detalhados de exceções e erros críticos por meio de um middleware global de tratamento, localizado em **server/src/middlewares/errorHandler.ts**. Para monitorar o comportamento do servidor em tempo real em busca de *bugs* ou gargalos durante a esteira de desenvolvimento, o programador deve analisar a saída padrão do terminal ativada pelo comando de inicialização em modo de observação **cd server && pnpm dev**.

Atualização tecnológica e gestão de dependências:

Estabelecimento de um cronograma periódico para a revisão e atualização das bibliotecas e frameworks utilizados no projeto (Next.js, Node.js e Prisma ORM). Essa prática é indispensável para corrigir vulnerabilidades de segurança descobertas no mercado e garantir a compatibilidade com servidores modernos. O mapeamento completo das dependências do ecossistema deve ser verificado nos arquivos **package.json** localizados nas raízes das pastas **server/** e **client/**. Para auditar o projeto em busca de pacotes obsoletos ou vulneráveis, o desenvolvedor, inicialmente, deve rodar o comando **pnpm outdated** ou **pnpm audit** nos respectivos diretórios, aplicando as atualizações necessárias de forma homologada.

Como escalar a solução?

O planejamento para a escalabilidade do sistema busca preparar a plataforma para suportar o crescimento do volume de dados, acervos e usuários ativos, sem que isso resulte em perda de performance ou elevação imprevisível dos custos de atualização da infraestrutura. Para garantir que o sistema evolua de uma aplicação local para uma solução de grande porte de maneira sustentável, estabelecem-se as seguintes diretrizes de expansão:

Otimização nas consultas:

À medida que a base de usuários cresce, o banco de dados tende a realizar varreduras completas e mais prolongadas para encontrar registros simples, como o histórico de empréstimos de um leitor. A indexação resolve esse problema ao criar uma estrutura de busca rápida, reduzindo drasticamente o uso de processamento e evitando que consultas simultâneas gerem travamentos ou lentidão generalizada no sistema. Dessa forma, a inclusão do comando **@@index** nos modelos do arquivo de configuração **server/prisma/schema.prisma** acelera as buscas operacionais e os relatórios analíticos. Os campos prioritários definidos para essa otimização são **titulo**, **autor** e **isbn** na entidade Livro; **nomeCliente** e **emailCliente** na entidade Empréstimo ; além do status combinado à data de vencimento para acelerar o cálculo de regras de negócio, consolidados localmente por meio da execução do comando **cd server && npx prisma migrate dev --name add_search_indexes**.

Implementação de autenticação:

A premissa de acesso livre sem autenticação por senha atende apenas a cenários locais e controlados, tornando-se o principal impedimento para a expansão segura do sistema. Para escalar a aplicação comercialmente ou até disponibilizá-la na internet, a ausência de barreiras de identidade gera riscos de vazamento de dados de leitores e manipulação indevida do acervo por agentes externos. Para mitigar essas vulnerabilidades, a implementação de criptografia e autenticação torna-se um pré-requisito obrigatório para garantir a proteção de dados administrativos e a segurança jurídica necessárias para a expansão do software. Toda essa lógica de controle deve ser estruturada por meio de tokens JWT, centralizados no arquivo **server/src/middlewares/authMiddleware.ts** para a proteção das rotas de controle e adaptação dos serviços associados. Tecnicamente, a integração desse mecanismo exige a adição dos pacotes necessários ao ambiente do servidor através do comando **cd server && npm add jsonwebtoken bcryptjs**.

Ampliação dos Canais de Comunicação:

O envio exclusivo de e-mails para a cobrança de atrasos de empréstimos apresenta baixas taxas de abertura no mercado geral e sofre frequentemente com barreiras de filtros de spam. Para sustentar uma alta rotatividade de empréstimos à medida que o ecossistema expande seu volume de leitores, a comunicação precisa ser direta e instantânea, reduzindo o descumprimento e os custos com servidores SMTP corporativos. Diante disso, a infraestrutura atual evoluirá para integrar o sistema a notificações enviadas diretamente na tela do celular do leitor e disparos automáticos via API oficial de mensagens instantâneas. Toda essa lógica de distribuição de alertas e expansão de rotas será desenvolvida centralizadamente no arquivo de serviços em **server/src/services/notificationServices.ts**, permitindo que o sistema amplie o alcance das mensagens sem interromper os fluxos de comunicação já consolidados.

Automação de Implantação e Testes:

À medida que a quantidade de desenvolvedores que passarão a atuar no projeto expande, o risco de novas funcionalidades introduzirem falhas em regras críticas aumenta exponencialmente. A automação de Integração e Entrega Contínuas (CI/CD) blinda a aplicação por meio da implementação de automações integradas ao repositório de código (como o GitHub Actions), obrigando que toda alteração submetida pela equipe de desenvolvimento passe por uma bateria rigorosa de verificações de segurança e testes computadorizados antes de ser enviada e publicada no ambiente de produção. Essa prática garante que o sistema nunca sofra interrupções por erro humano em ambiente produtivo. Para que essa validação também seja realizada de forma prévia e local antes do envio ao repositório, os desenvolvedores devem executar os testes automatizados tanto no ecossistema do servidor quanto no cliente, utilizando os comandos **cd server && npm test** e **cd client && npm test**.

Informações ao usuário

As informações exibidas no sistema dependem dos dados cadastrados previamente. Por isso, caso algum livro, nome de usuário ou empréstimo não apareça corretamente, é possível que essas informações ainda não tenham sido registradas ou atualizadas no sistema.

Na versão web, o cadastro de livros e empréstimos deve ser realizado com atenção, pois esses dados serão utilizados nas consultas e visualizações disponíveis no sistema. Informações incompletas ou incorretas podem afetar os resultados apresentados tanto para administradores quanto para leitores.

Na versão mobile, as consultas por livro ou por nome de usuário retornam os empréstimos associados aos termos pesquisados. Caso a busca não apresente resultados, é recomendado verificar se o nome foi digitado corretamente ou se o empréstimo realmente está cadastrado no sistema.

As datas de empréstimo e devolução exibidas no aplicativo seguem as informações registradas no momento do cadastro do empréstimo. Como a alteração dos dados de um empréstimo ainda não está disponível no sistema, é importante conferir as informações antes de concluir o cadastro.

Para utilizar o sistema corretamente, é importante manter uma conexão estável com a internet. Caso alguma tela não carregue ou as informações não sejam exibidas, recomenda-se atualizar a página, reiniciar o aplicativo ou tentar novamente após alguns instantes. Em caso de dados incorretos, ausência de informações ou dificuldades de acesso, o usuário deve entrar em contato com o responsável pela administração do sistema.

Tecnologias utilizadas

O projeto foi desenvolvido utilizando diferentes tecnologias para atender às três principais partes da aplicação: a interface web, o aplicativo mobile e o backend. Cada uma dessas tecnologias contribui para a construção das telas, comunicação com o servidor, organização dos dados e funcionamento geral do sistema.

- **TypeScript:** O TypeScript foi utilizado como linguagem principal do projeto. Ele permite escrever o código com tipagem, o que ajuda a evitar erros durante o desenvolvimento e torna mais claro o formato dos dados utilizados na aplicação. Está presente tanto na versão web quanto no mobile e no backend, ajudando a manter maior consistência entre as diferentes partes do sistema;
- **Next.js:** O Next.js foi utilizado no desenvolvimento da interface web da aplicação. Ele é responsável pela estrutura das páginas acessadas pelos administradores, como o Dashboard, a tela de Livros e a tela de Novo Livro;
- **React:** O React é a base utilizada na construção da interface web. Ele permite criar componentes reutilizáveis, como botões, campos, modais, cards e tabelas. Essa organização por componentes ajudou a manter as telas mais padronizadas e facilitou a construção das funcionalidades usadas pelos administradores;
- **React Native:** O React Native foi utilizado no desenvolvimento da aplicação mobile. Ele permite criar telas e componentes próprios para dispositivos móveis, utilizando uma estrutura semelhante à do React. Foi usado para construir a tela principal do aplicativo, os cards de empréstimos, os campos de busca e os elementos visuais exibidos no celular;
- **Expo:** O Expo foi utilizado para facilitar o desenvolvimento e a execução da aplicação mobile. Com ele, é possível testar o aplicativo no celular por meio do Expo Go, sem a necessidade de configurar um ambiente mobile mais complexo logo no início. Também permite que os desenvolvedores acompanhem as alterações diretamente no dispositivo durante o desenvolvimento;
- **Tailwind CSS:** O Tailwind CSS foi utilizado principalmente na estilização da interface web e também serve de base para o uso do NativeWind no mobile. Ele permite aplicar estilos de forma prática, diretamente nos componentes. Sua utilização ajudou a manter uma identidade visual mais consistente e facilitou a construção das telas administrativas;

- **NativeWind:** O NativeWind foi utilizado na estilização da interface mobile. Ele permite aplicar estilos aos componentes do React Native de forma parecida com o Tailwind CSS. Essa tecnologia ajudou na organização visual do aplicativo, facilitando ajustes de espaçamento, alinhamento, tamanhos e aparência dos elementos exibidos na tela;
- **Shadcn UI:** O Shadcn UI foi utilizado na interface web para apoiar a construção de componentes visuais. Ele oferece componentes prontos e adaptáveis, que podem ser ajustados conforme a necessidade da aplicação. Sua utilização contribuiu para a criação de uma interface mais organizada, especialmente em elementos como botões, campos, modais e estruturas de interação usadas pelos administradores;
- **Radix UI:** O Radix UI aparece como base para alguns componentes utilizados na interface web, especialmente componentes que precisam lidar com interações, acessibilidade e comportamento visual, como diálogos, labels, separadores e áreas clicáveis;
- **Axios:** O Axios foi utilizado para realizar a comunicação entre as interfaces e o backend. Ele permite que a versão web e a versão mobile busquem ou enviem informações para o servidor. Essa comunicação é necessária para ações como listar livros, buscar empréstimos, cadastrar informações e atualizar dados exibidos nas telas;
- **Node.js:** O Node.js foi utilizado no backend da aplicação. Ele permite executar o servidor responsável por processar as requisições feitas pela versão web e pelo aplicativo mobile. O backend concentra regras importantes do sistema, como o cadastro de livros, registro de empréstimos, controle de devoluções e identificação de atrasos;
- **Express:** O Express foi utilizado junto ao Node.js para estruturar a API do sistema. Ele ajuda a organizar as rotas que recebem e respondem às solicitações feitas pelas interfaces. Essas rotas permitem que a aplicação consulte, cadastre, atualize e remova informações relacionadas aos livros e empréstimos;
- **Prisma ORM:** O Prisma ORM foi utilizado para facilitar a comunicação entre o backend e o banco de dados. Ele permite representar as entidades do sistema, como livros e empréstimos, de forma mais organizada no código. O Prisma ajuda a realizar operações no banco de dados com mais segurança e clareza, reduzindo a necessidade de escrever consultas manuais diretamente;

- **PostgreSQL:** O PostgreSQL é o banco de dados utilizado para armazenar as informações do sistema. Nele ficam registrados dados como livros, categorias, usuários e empréstimos;
- **Docker:** O Docker foi utilizado para facilitar a execução do ambiente do projeto, principalmente em relação ao banco de dados. Com ele, é possível subir os serviços necessários de forma mais padronizada entre os membros da equipe. O Docker ajuda a reduzir diferenças de configuração entre máquinas, tornando o ambiente de desenvolvimento mais estável;
- **Nodemailer:** O Nodemailer foi utilizado no backend para o envio de e-mails. Essa tecnologia permite enviar notificações relacionadas a empréstimos em atraso. Com isso, o administrador pode acionar o envio de aviso ao usuário responsável, ajudando no acompanhamento das devoluções pendentes;
- **Node-cron:** O Node-cron foi utilizado para lidar com tarefas automáticas no backend. Ele permite agendar execuções em determinados momentos, sem depender de uma ação manual constante. Essa tecnologia foi utilizada para apoiar rotinas relacionadas à verificação de atrasos e envio de notificações;
- **Fuse.js:** O Fuse.js foi utilizado para auxiliar em buscas mais flexíveis. Essa tecnologia permite encontrar resultados mesmo quando a pesquisa não corresponde exatamente ao texto cadastrado. Seu uso contribuiu para melhorar a experiência de consulta, especialmente em buscas por livros ou usuários;
- **Recharts:** O Recharts foi utilizado na interface web para a criação dos gráficos exibidos no Dashboard. Ele permite transformar os dados da biblioteca em visualizações mais fáceis de interpretar. Essa tecnologia possibilitou a apresentação de gráficos como quantidade de livros por categoria, quantidade de empréstimos por categoria e livros com maior número de empréstimos ativos;
- **Lucide React e Lucide React Native:** O Lucide foi utilizado para adicionar ícones às interfaces do sistema. Na versão web, é utilizado com React, enquanto na versão mobile é utilizado com React Native;
- **Zod:** O Zod foi utilizado para validação de dados. Ele ajuda a conferir se as informações seguem o formato esperado antes de serem utilizadas pela aplicação. Essa validação contribui para reduzir erros em campos e dados enviados pelo usuário;

- **Jest:** O Jest foi utilizado como ferramenta de testes. Ele permite verificar se partes do código estão funcionando corretamente. Essa tecnologia ajuda a aumentar a confiabilidade da aplicação durante o desenvolvimento;
- **Testing Library:** A Testing Library foi utilizada em conjunto com o Jest para apoiar testes nas interfaces. Ela permite testar componentes considerando o comportamento esperado pelo usuário. Essa ferramenta auxilia na verificação de componentes da aplicação web e mobile;
- **PNPM:** O pnpm foi utilizado como gerenciador de pacotes do projeto. Ele é responsável por instalar e organizar as dependências necessárias para executar cada parte da aplicação. O pnpm é usado nos comandos de instalação e execução do backend, da interface web e do aplicativo mobile.

Arquitetura do projeto

O projeto foi desenvolvido aplicando os princípios da Clean Architecture, promovendo a separação das responsabilidades entre as camadas da aplicação. A solução foi **dividida em três módulos principais**, sendo eles **Frontend (Client)**, **Backend (Server)** e **Mobile**. Essa organização tem como objetivo facilitar a manutenção, a realização de testes e a escalabilidade do sistema.

Frontend (client)

```
client
├── public
│   └── Capas de Livros
├── src
│   ├── @types
│   ├── app
│   ├── assets
│   ├── components
│   ├── hooks
│   ├── lib
│   ├── services
│   └── styles
└── tests
```

Backend (server)

```
server
├── prisma
│   └── migrations
├── src
│   ├── controllers
│   ├── database
│   ├── dtos
│   ├── errors
│   ├── jobs
│   ├── middlewares
│   ├── repositories
│   ├── routes
│   ├── services
│   └── utils
├── tests
│   ├── backendTests
│   └── integrationTests
```




mobile

```
mobile
├── app
├── src
│   ├── assets
│   ├── components
│   ├── services
│   └── styles
└── __tests__
```

Estrutura e organização do projeto

1. client/

Trata-se do diretório raiz da aplicação frontend. Centraliza todos os recursos relacionados à interface do sistema, incluindo código-fonte, arquivos estáticos, estilos, serviços de comunicação com a API e testes automatizados.

1.1. public/

Armazena arquivos estáticos disponibilizados diretamente para o navegador, sem necessidade de processamento durante o processo de compilação da aplicação.

1.1.1. Capas de Livros/

Contém as imagens utilizadas como capas dos livros por categoria. Esses arquivos são utilizados para representação visual do acervo e são carregadas diretamente pela interface.

1.2. src/

Contém todo o código-fonte da aplicação frontend. É responsável pela implementação da interface, lógica de apresentação, comunicação com serviços externos e organização dos recursos utilizados pelo sistema.

1.2.1. @types/

Armazena definições de tipos e interfaces utilizadas pelo typescript. O objetivo é padronizar estruturas de dados, aumentar a segurança de tipagem e facilitar a manutenção do código.

1.2.2. app/

Responsável pela organização das páginas, rotas e fluxos principais da aplicação. Centraliza a estrutura de navegação e os componentes que representam as telas acessadas pelos usuários.

1.2.3. assets/

Agrupar recursos utilizados pela interface que precisam ser processados pelo sistema de build, como imagens, ícones, fontes e outros arquivos de apoio visual.

1.2.4. components/

Contém os componentes reutilizáveis da interface. Esses componentes representam elementos visuais ou funcionais que podem ser utilizados em diferentes páginas da aplicação, promovendo a padronização.

1.2.5. hooks/

Armazena hooks personalizados responsáveis por encapsular lógicas reutilizáveis relacionadas ao gerenciamento de estado, manipulação de dados, chamadas assíncronas e comportamentos compartilhados entre componentes.

1.2.6. lib/

Reúne configurações, utilitários e bibliotecas auxiliares utilizadas em diferentes partes da aplicação. Contém abstrações, funções genéricas e integrações necessárias para o funcionamento do sistema.

1.2.7. services/

Responsável pela comunicação com os serviços externos, especialmente a API backend. Contém funções encarregadas de realizar requisições, enviar dados, receber respostas e tratar operações relacionadas ao consumo de serviços.

1.2.8. styles/

Centraliza o arquivo de estilização da aplicação, incluindo definições globais de aparência, variáveis visuais, temas e regras responsáveis pela identidade visual do sistema.

1.3. tests/

Contém os testes automatizados da aplicação frontend. É responsável por validar o comportamento dos componentes, páginas e demais funcionalidades, garantindo a estabilidade e a confiabilidade do sistema durante seu desenvolvimento e manutenção.

Possui esses arquivos específicos **Badge.test.tsx**, **BookCard.test.tsx**, **BookDetailModal.test.tsx**, **BookFiltersModal.test.tsx**, **Button.test.tsx**, **CategoryTest.teste.tsx**, **LoanHistoryCard.test.tsx**, **LoanModal.test.tsx**, **StatCard.test.tsx** e cada arquivo corresponde a um componente específico do sistema, mantendo a organização e facilitando a identificação de problemas durante o desenvolvimento.

Principais funcionalidades da pasta:

- verificar se os componentes exibem corretamente as informações recebidas, como títulos, nomes, status e quantidades;
- garantir que botões e campos de formulário respondem às interações do usuário da forma esperada;
- validar o comportamento condicional dos componentes, como elementos que aparecem ou somem dependendo do estado recebido;
- testar a exibição de mensagens de erro, carregamento e estados vazios;
- garantir que as funções associadas a ações do usuário são chamadas corretamente ao interagir com a interface;
- preparar o ambiente de testes para simular recursos do navegador que não estão disponíveis no ambiente de execução do Jest.

2. server/

Diretório raiz da aplicação backend. Reúne todo o código-fonte, configurações, banco de dados, migrações e testes responsáveis pelas regras de negócio e pela disponibilização dos serviços da aplicação.

2.1. prisma/

Responsável pela configuração e gerenciamento do banco de dados através do ORM Prisma. Centraliza definições relacionadas à persistência dos dados e ao versionamento da estrutura do banco. Através da pasta migration, armazena o histórico de migrações do banco de dados. Cada migração representa um conjunto de alterações estruturais, como criação, modificação ou remoção de tabelas, colunas e relacionamentos.

2.2. src/

Concentra os códigos referentes a aplicação, incluindo regras de negócio, rotas, validações, integração com o banco de dados e componentes auxiliares.

2.2.1. controllers/

Reúne os arquivos responsáveis por receber as requisições HTTP, interpretar os dados enviados pelo cliente e os encaminhar para os serviços apropriados. Também define as respostas retornadas pela API. Os arquivos relacionados a esta camada são: [bookController.ts](#), [dashboardController.ts](#), [loanController.ts](#).

2.2.2. database/

Centraliza as configurações e recursos relacionados à conexão e comunicação com o banco de dados, servindo de acesso para operações de persistência.

2.2.3. dtos/back

Armazena os Data Transfer Objects (DTOs), estruturas utilizadas para padronizar e validar os dados trafegados entre as diferentes camadas da aplicação.

2.2.4. errors/

Contém as classes e definições de erros personalizados utilizados pelo sistemas. Padronizar mensagens de erros e códigos de resposta em diferentes cenários de exceção.

2.2.5. jobs/

A pasta jobs/ reúne arquivos responsáveis por tarefas automáticas executadas pelo backend. Essas tarefas não dependem diretamente de uma ação manual do usuário em uma tela, pois são criadas para rodar em segundo plano em momentos definidos pela aplicação.

No contexto do sistema de biblioteca, essa pasta está relacionada a rotinas que ajudam no acompanhamento dos empréstimos e atrasos. Ela pode ser utilizada, por exemplo, para verificar empréstimos vencidos e apoiar o envio de notificações aos usuários responsáveis.

Essa organização é importante porque separa tarefas automáticas das rotas e regras acionadas diretamente pelas interfaces web ou mobile. Dessa forma, o backend consegue manter rotinas internas funcionando de maneira mais organizada.

Principais responsabilidades da pasta:

- executar rotinas automáticas do backend;
- apoiar a verificação de empréstimos em atraso;
- organizar tarefas que podem rodar em segundo plano;
- separar processamentos automáticos das requisições feitas pelas telas;
- facilitar a manutenção de funcionalidades recorrentes ou agendadas.

2.2.6. middlewares/

Armazena componentes intermediários executados durante o processamento das requisições. São utilizados para validação, autenticação, autorização, tratamento de erros e outras verificações necessárias antes da execução das regras de negócio.

2.2.7. repositories/

Responsável pelo acesso aos dados persistidos. Implementa operações de consulta, criação, atualização e remoção de registros, abstraindo a comunicação direta com o banco de dados. Foram criados o [loanRepository.ts](#),

2.2.8. routes/

Define os endpoints disponibilizados pela API e realiza o mapeamento entre as rotas da aplicação e seus respectivos controladores.

2.2.9. services/

Contém as regras de negócio da aplicação. É responsável por processar informações, aplicar validações, executar operações complexas e coordenar a comunicação entre controladores e repositórios. Foram implementados arquivos services para as funcionalidades referentes a livros, empréstimos e dashboard, de maneira que os requisitos de funcionalidade fossem atendidos e validados.

2.2.10. utils/

A pasta utils armazena arquivos auxiliares utilizados pelo backend. Seu objetivo é concentrar lógicas de apoio que podem ser reutilizadas por diferentes partes do sistema, evitando repetição de código e facilitando a manutenção. No sistema, essa pasta é utilizada para reunir comportamentos relacionados ao tratamento de informações auxiliares, como a definição de status de empréstimos e a padronização visual de elementos usados em notificações.

statusHelpers.ts:

Arquivo responsável por centralizar funcionalidades auxiliares relacionadas ao status dos empréstimos. Ele é utilizado para identificar a situação de um empréstimo a partir da data prevista de devolução, indicando se o empréstimo ainda está em

andamento ou se já se encontra em atraso. Além disso, esse arquivo também auxilia na definição das cores utilizadas para representar visualmente cada status nos e-mails enviados pelo sistema. Dessa forma, as informações de status ficam mais padronizadas e fáceis de identificar nas notificações.

Principais funcionalidades desse arquivo são:

- definir o status de um empréstimo com base na data prevista de devolução;
- diferenciar empréstimos em andamento de empréstimos atrasados;
- apoiar a padronização visual dos status exibidos nos e-mails;
- centralizar regras auxiliares relacionadas ao status dos empréstimos.

2.3. tests/

Contém os testes automatizados responsáveis por verificar o funcionamento correto da aplicação e garantir a confiabilidade das funcionalidades implementadas.

2.3.1. integrationTests/

Contém os arquivos que testam a integração entre o frontend e o backend, cada um configurado com suas respectivas funções que visam testar o comportamento ideal da aplicação, sem impactar no banco de dados real.

2.3.1.1.

2.3.2. backendTests/

Contém os testes de validação das regras de negócio dos serviços de book, loan e dashboard, garantindo que o comportamento da aplicação esteja de acordo com o esperado de forma isolada, sem impacto no banco de dados real.

3. mobile/

O diretório mobile reúne os arquivos responsáveis pela aplicação mobile do sistema. Essa área é voltada para a consulta de empréstimos pelo celular, permitindo que o usuário visualize informações relacionadas aos livros, usuários e datas de empréstimo e devolução. A estrutura do mobile é organizada em telas, componentes reutilizáveis,

serviços de comunicação com o backend, arquivos de estilo, recursos visuais e testes automatizados.

3.1. Tests/

A pasta `__Tests__` reúne os testes automatizados da aplicação mobile. Esses testes são utilizados para verificar se os principais componentes da interface estão sendo renderizados corretamente e se continuam funcionando após alterações no código.

LoanCard.test.tsx:

Arquivo de teste do componente `LoanCard`. Ele verifica se o card de empréstimo exibe corretamente as informações esperadas, como dados do usuário, livro relacionado, capa, data de empréstimo e data de devolução;

MobileHeader.test.tsx:

Arquivo de teste do componente `MobileHeader`. Ele verifica se o cabeçalho da aplicação mobile é exibido corretamente, garantindo que a estrutura visual inicial da tela esteja funcionando como esperado;

SearchButton.test.tsx:

Arquivo de teste do componente `SearchButton`. Ele valida se o botão de busca é renderizado corretamente e se pode ser utilizado na interação de pesquisa da aplicação;

SearchInput.test.tsx:

Arquivo de teste do componente `SearchInput`. Ele verifica se o campo de busca aparece corretamente na tela e se está preparado para receber o texto digitado pelo usuário durante a consulta.

3.2. app/

A pasta `app/` contém a estrutura principal de telas e navegação da aplicação mobile. Como o projeto utiliza Expo Router, essa pasta define quais as telas que fazem parte da aplicação e como elas são carregadas.

index.tsx:

Arquivo responsável pela tela principal do aplicativo mobile. Nele, o usuário consegue visualizar os empréstimos e realizar pesquisas por livro ou por nome de usuário. Essa tela utiliza os componentes criados na pasta `componentes/`, como o cabeçalho, campo de busca, botão de

busca e cards de empréstimo. Também se comunica com os serviços da aplicação para buscar os dados necessários no backend.

As principais responsabilidades desse arquivo são:

- exibir a tela principal do aplicativo;
- controlar o texto digitado no campo de busca;
- realizar a consulta de empréstimos;
- atualizar os resultados exibidos na tela;
- organizar a exibição dos cards de empréstimos para o usuário;

_layout.tsx:

Arquivo responsável por definir a estrutura base da aplicação mobile. Ele funciona como o ponto de configuração da navegação utilizada pelo Expo Router.

Esse arquivo garante que a tela principal seja carregada dentro da estrutura esperada pelo aplicativo e ajuda a organizar o fluxo de navegação da aplicação.

3.3. src/

A pasta `src/` concentra os arquivos de apoio utilizados pela aplicação mobile. Nela ficam os recursos visuais, componentes reutilizáveis, serviços de comunicação e arquivos de estilo.

Essa divisão facilita a manutenção do código, pois separa as responsabilidades da aplicação em áreas específicas.

3.3.1. assets/

A pasta `assets` armazena os recursos visuais utilizados no aplicativo mobile. Esses arquivos são usados para complementar a interface e melhorar a apresentação das informações ao usuário.

capasLivros/:

Pasta responsável por armazenar as capas dos livros utilizados na aplicação. Essas imagens são exibidas nos cards de empréstimo, permitindo que o usuário identifique visualmente o livro relacionado ao empréstimo;

icons/:

Pasta responsável por armazenar ícones utilizados na interface mobile. Esses ícones ajudam a representar visualmente ações ou informações dentro do aplicativo;

index.ts:

Arquivo responsável por organizar e exportar os recursos visuais da pasta assets. Ele facilita a importação das imagens e ícones em outros arquivos da aplicação, evitando caminhos longos ou repetidos.

3.3.2. componentes/

A pasta componentes reúne os componentes reutilizáveis da interface mobile. Esses componentes são partes menores da tela que podem ser combinadas para formar a experiência completa do aplicativo. A utilização de componentes ajuda a manter o código mais organizado, evita repetição e facilita ajustes visuais ou funcionais.

LoanCard/

Pasta responsável pelo componente que exibe as informações de um empréstimo. O LoanCard apresenta os dados principais de cada empréstimo, como nome do usuário, livro relacionado, capa do livro, data de empréstimo e data de devolução. Ele organiza essas informações em formato de card, tornando a consulta mais clara para o usuário.

Principais responsabilidades do componente:

- receber os dados de um empréstimo;
- exibir as informações do livro e do usuário;
- mostrar a capa do livro quando disponível;
- apresentar as datas de empréstimo e devolução;
- organizar visualmente cada empréstimo na lista exibida no aplicativo;

MobileHeader/

Pasta responsável pelo componente de cabeçalho da aplicação mobile. O MobileHeader é utilizado para apresentar a identificação visual inicial do aplicativo, ajudando a compor a estrutura da tela principal. Ele contribui para que a interface tenha uma apresentação mais organizada e reconhecível.

Principais responsabilidades do componente:

- exibir o cabeçalho da aplicação;
- contribuir para a identidade visual da tela mobile;
- separar a área superior da aplicação do restante do conteúdo;

SearchButton/

Pasta responsável pelo componente de botão utilizado na busca. O SearchButton é acionado quando o usuário deseja realizar uma consulta.

Ele trabalha junto com o campo de busca, permitindo pesquisar empréstimos associados a um livro ou a um usuário.

Principais responsabilidades do componente:

- exibir o botão de pesquisa;
- permitir que o usuário inicie uma busca;
- acionar a função responsável por consultar os dados;
- reforçar visualmente a ação de pesquisa dentro da interface;

SearchInput/

Pasta responsável pelo componente de campo de busca. O SearchInput permite que o usuário digite o nome de um livro ou o nome de um usuário para consultar os empréstimos associados. Ele é uma das principais formas de interação do usuário com o aplicativo.

Principais responsabilidades do componente:

- receber o texto digitado pelo usuário;
- permitir consultas por livro;
- permitir consultas por nome de usuário;
- trabalhar em conjunto com o botão de busca;
- ajudar a filtrar os empréstimos exibidos na tela.

3.3.3. Services/

A pasta Services concentra os arquivos responsáveis pela comunicação entre o aplicativo mobile e o backend da aplicação. Essa camada evita que as chamadas para a API fiquem espalhadas diretamente pelas telas ou componentes. Com isso, a comunicação com o servidor fica centralizada e mais fácil de manter.

api.ts:

Arquivo responsável por configurar a comunicação do aplicativo mobile com a API do sistema. Nesse arquivo, normalmente são definidas configurações como o endereço base do servidor e a instância utilizada para realizar requisições. A partir dele, a aplicação consegue buscar informações relacionadas aos empréstimos, livros e usuários cadastrados.

Principais responsabilidades do arquivo:

- centralizar a configuração de comunicação com o backend;
- permitir que o mobile faça requisições para a API;
- facilitar a busca de dados utilizados na tela principal;
- apoiar a atualização das informações exibidas no aplicativo.

3.3.4. style/

A pasta style reúne arquivos relacionados à estilização e aparência da aplicação mobile. Esses arquivos ajudam a manter a identidade visual do aplicativo e a organizar padrões utilizados em diferentes partes da interface.

global.css:

Arquivo responsável por estilos globais da aplicação. Ele pode conter configurações gerais utilizadas em toda a interface mobile, servindo como base para a aparência do aplicativo;

theme.ts:

Arquivo responsável por centralizar definições visuais do tema da aplicação, como cores, padrões de estilo e outras configurações reutilizáveis. Esse arquivo ajuda a manter consistência visual entre os componentes, evitando que valores de estilo sejam repetidos em várias partes do código.

Links relevantes

Durante o desenvolvimento da solução, foram utilizadas documentações oficiais e materiais de apoio relacionados às tecnologias presentes no projeto. Esses links serviram como referência para entender melhor as ferramentas, consultar exemplos de uso e resolver dúvidas durante a implementação.

- **TypeScript:**
[A Documentação oficial do TypeScript](#) – utilizada como referência para tipagem e organização dos dados da aplicação;
- **Next.js:**
[Documentação oficial do Next.js](#) – utilizada como apoio para a estruturação da interface web e organização das páginas da aplicação;
- **React:**
[Documentação oficial do React](#) – utilizada como referência para criação de componentes e organização da interface web;
- **React Native:**
[Documentação oficial do React Native](#) – utilizada como apoio para o desenvolvimento da aplicação mobile;
- **Expo:**
[Documentação oficial do Expo](#) – utilizada como referência para execução, testes e configuração da aplicação mobile;
- **Expo Go:**
[Página oficial do Expo Go](#) – utilizada como apoio para testar o aplicativo mobile diretamente no celular;
- **Tailwind CSS:**
[Documentação oficial do Tailwind CSS](#) – utilizada como referência para estilização da interface web;
- **NativeWind:**
[Documentação oficial do NativeWind](#) – utilizada como referência para aplicar estilos no aplicativo mobile com uma lógica semelhante ao Tailwind CSS;

- **Shadcn UI:**
[Documentação oficial do Shadcn UI](#) — utilizada como referência para componentes visuais da interface web;
- **Radix UI:**
[Documentação oficial do Radix UI](#) — utilizada como referência para componentes acessíveis e estruturas de interação da interface;
- **Axios:**
[Documentação oficial do Axios](#) — utilizada como referência para comunicação entre as interfaces e o backend;
- **Node.js:**
[Documentação oficial do Node.js](#) — utilizada como referência para o ambiente de execução do backend;
- **Express:**
[Documentação oficial do Express](#) — utilizada como referência para estruturação das rotas e da API do sistema;
- **Prisma ORM:**
[Documentação oficial do Prisma ORM](#) — utilizada como referência para comunicação entre o backend e o banco de dados;
- **PostgreSQL:**
[Documentação oficial do PostgreSQL](#) — utilizada como referência para o banco de dados da aplicação;
- **Docker:**
[Documentação oficial do Docker](#) — utilizada como referência para configuração e execução do ambiente de desenvolvimento;
- **Nodemailer:**
[Documentação oficial do Nodemailer](#) — utilizada como referência para o envio de e-mails pelo backend;
- **Node-cron:**
[Documentação do Node-cron no npm](#) — utilizada como referência para agendamento de tarefas automáticas no backend;

- **Fuse.js:**
[Documentação oficial do Fuse.js](#) – utilizada como referência para buscas mais flexíveis na aplicação;
- **Recharts:**
[Documentação oficial do Recharts](#) – utilizada como referência para construção dos gráficos exibidos no Dashboard;
- **Lucide React:**
[Documentação oficial do Lucide React](#) – utilizada como referência para uso de ícones na interface web;
- **Lucide React Native:**
[Documentação oficial do Lucide React Native](#) – utilizada como referência para uso de ícones na aplicação mobile;
- **Zod:**
[Documentação oficial do Zod](#) – utilizada como referência para validação de dados;
- **Jest:**
[Documentação oficial do Jest](#) – utilizada como referência para testes da aplicação;
- **Testing Library:**
[Documentação oficial da Testing Library](#) – utilizada como apoio para testes voltados ao comportamento esperado pelo usuário;
- **pnpm:**
[Documentação oficial do pnpm](#) – utilizada como referência para instalação e gerenciamento das dependências do projeto.

Comentários

Lucas Fernandes:

O processo de criação da solução foi uma experiência muito marcante, principalmente por permitir acompanhar a evolução do projeto desde as primeiras entregas até a construção de funcionalidades mais completas. Foi muito interessante perceber como pequenas implementações, como a criação de um botão, serviram de base para etapas maiores, como modais, telas, fluxos de cadastro e funcionalidades integradas.

Durante o processo, também foi importante perceber que as atividades foram bem direcionadas. Isso facilitou o entendimento do que precisava ser feito em cada etapa e ajudou o squad a manter uma boa organização ao longo do desenvolvimento. A cada nova task concluída, ficava mais evidente a evolução da equipe, tanto no conhecimento técnico quanto na forma de trabalhar em conjunto.

Em relação ao PTA, mesmo com alguns desafios, como questionamentos sobre o uso de um boilerplate desatualizado, a experiência como um todo foi muito positiva. Os momentos de working day, "Call de Dev" e encontros coletivos contribuíram bastante para criar um sentimento de pertencimento ao CITi, mesmo que ainda na posição de trainee. Essas vivências tornaram o processo mais próximo, colaborativo e humano.

Um dos pontos mais interessantes do PTA foi a forma como ele possibilitou a conexão com diferentes pessoas ao longo do ciclo. Em muitos momentos, pessoas que já estavam próximas acabaram se tornando descobertas muito positivas, seja pela convivência, pela troca de conhecimento ou pela parceria durante o desenvolvimento. Isso tornou a experiência mais leve e significativa.

Sobre o squad, é difícil imaginar um grupo mais unido. A equipe esteve presente em diversos momentos, tanto nas entregas quanto nas interações fora das tasks, seja para almoçar, conversar, tirar dúvidas ou até realizar working days online. Essa união fez diferença no andamento do projeto e contribuiu para que o trabalho acontecesse de forma mais colaborativa.

A cada nova solução entregue, foi possível admirar ainda mais o empenho da equipe. Todos demonstraram evolução ao longo do processo, cada um à sua forma, e isso ficou evidente na qualidade das entregas e na estabilidade do progresso. Ao todo, foram 34 tasks concluídas, além de outros ajustes e melhorias realizados durante o desenvolvimento. Demonstrando o comprometimento do grupo e a capacidade de manter uma boa organização mesmo diante dos desafios.

Em relação ao gerente, a experiência também foi muito positiva. O squad foi bem orientado durante todo o processo, o que contribuiu diretamente para o progresso das entregas e para a segurança da equipe ao desenvolver as funcionalidades. As orientações recebidas ajudaram a manter o direcionamento do projeto e permitiram que as dúvidas fossem resolvidas de forma clara.

Mesmo considerando as dificuldades naturais do final de período, com provas, projetos e outras atividades acadêmicas, em nenhum momento houve uma sensação de desamparo. Pelo contrário, a equipe conseguiu seguir avançando com apoio, autonomia e confiança.

De modo geral, o processo foi desafiador, mas também muito enriquecedor, tanto pelo aprendizado técnico quanto pela experiência de colaboração, convivência e construção coletiva.

Matheus Agra:

Trabalhar para a entrega dessa solução proporcionou à equipe uma experiência extremamente enriquecedora, principalmente por permitir se redescobrir mais e mais cada vez que era preciso mergulhar a fundo no processo de desenvolvimento. Ao passar por todas as etapas do projeto, desde a idealização do produto à revisão do que já estava definido para ser entregue ao cliente, foi possível conhecer diferentes tecnologias presentes no mercado atual e aplicar o conhecimento previamente adquirido.

O ambiente de trabalho leve e inspirador foi construído por cada um dos participantes, que demonstraram intensidade já no primeiro dia do programa. E o resultado disso é entender que cada mensagem trocada e cada retorno dado pelos companheiros não eram apenas práticas cotidianas de quem estava participando de mais um treinamento, mas sim frutos da vivência real de uma equipe que se doou constantemente para mostrar vontade de evoluir e trabalhar em busca de um objetivo em comum.

Para cada tarefa requisitada individualmente, era exigido da equipe pleno entendimento das tecnologias utilizadas, além de dedicar tempo para compreender o curso das tarefas dos companheiros, a fim de não criar conflitos de código. Cada componente do *frontend* passou, naturalmente, por uma análise minuciosa de dependências e de *design*; diante disso, surgiram diversas propostas de ideal funcionamento, que eram revisadas constantemente pelo Gerente de Software e aprovadas pelo cliente. Da mesma forma, para cada modelo do Prisma e operação que manipulava o banco de dados, era preciso ter o entendimento de como esses dados iriam ser apresentados ao cliente.

O PTA foi um programa de treinamento que pôs à prova o conhecimento e o desempenho do aspirante do início ao fim. As exigências semanais desenvolveram na equipe um senso de compromisso e de urgência, porque esse é um ambiente que transparece a realidade do desenvolvimento de um produto que será entregue a um cliente.

Para além das dificuldades enfrentadas durante o processo, como impedimentos pessoais e o dever de conciliar as obrigações universitárias com as demandas semanais, o processo seletivo fortaleceu pontos fracos, expôs forças e fez entender que se reinventar é sempre possível.

Maysa Barros:

O desenvolvimento da solução foi uma experiência ímpar. Acompanhar todo o processo, desde um botão criado na primeira sprint até todo o sistema pronto e funcionando, foi muito gratificante. Ao passar por todas as fases da criação, aprender como implementar cada funcionalidade e como funciona todo o ecossistema integrado, colaborou para conectar termos e resoluções técnicas a como são, mesmo que em menor escala, o dia a dia e a criação de projetos do CITi.

Nessas condições, é clara a maturidade adquirida por todos ao longo da execução da solução, melhorando, como exemplo, a agilidade em sinalizar e resolver os erros encontrados, além de todas as tasks enviadas com precisão e antecedência.

A integração do squad aconteceu naturalmente, unindo toda a vontade e a qualidade de cada integrante. Desde o início, todos se mostraram muito querentes e solícitos para se ajudar, demonstrando muito bem pontos essenciais para as resoluções de problemas como o espírito de time e a política do “Vai lá, Marca e Comemora” (enaltecendo todos os feitos do squad, independente do grau de complexidade da task). Juntos, participamos ativamente dos momentos do PTA, mas também, criamos um vínculo extra-profissional, que foi muito importante para o progresso do nosso projeto.

Sobre a gerência de Philip, foi de extrema importância ter um gerente tão eficaz e que nos puxasse tanto para mostrarmos e desenvolvermos o nosso melhor, quanto para nos ajudar das mais diversas maneiras. Desde a primeira sprint, tudo foi pensado por ele para trabalharmos, inicialmente, com partes do desenvolvimento que tínhamos mais dificuldade para aprimorarmos esses pontos e depois conseguirmos ir para o que já tínhamos uma certa afinidade. Essa metodologia foi essencial para desenvolvermos o sistema de uma forma fluida. Além disso, a integração do squad com o gerente foi realizada de maneira sincera, sempre valorizando o jogo limpo.

A experiência do PTA em si foi muito engrandecedora nos mais diversos âmbitos. Apesar de todos os desafios enfrentados no período acadêmico e no processo em si, a quantidade e a qualidade de tudo que foi aprendido se sobressai. Acabamos tendo alguns problemas, por conta do boilerplate desatualizado, porém, olhando pelo lado positivo, resolver isso juntos, acabou enriquecendo o nosso processo. As oportunidades de diversas maneiras de treinamento, momentos presenciais (como a Call de Dev, os Working Days e as Rodas) e remotos também tornaram tudo mais leve e especial. Tive a chance de conhecer pessoas que me inspiram e impulsionam a querer aprender e entender cada vez mais sobre esse mundo. A bagagem que adquiri, tanto técnica, quanto social, foi exponencial e importante para o meu desenvolvimento enquanto profissional.

Safira Moraes:

O desenvolvimento da solução foi uma experiência de grande aprendizado e crescimento. Ao longo do projeto, surgiu a oportunidade de atuar em diferentes frentes do desenvolvimento, o que permitiu ampliar conhecimentos e ganhar mais confiança em áreas nas quais ainda havia espaço para evolução. Um dos pontos mais marcantes dessa experiência foi o contato com testes unitários. A criação de testes para validar regras de negócio exigiu um olhar mais crítico sobre os requisitos do sistema, indo além de simplesmente desenvolver funcionalidades. Foi necessário questionar cenários, prever possíveis falhas e garantir que cada comportamento estivesse de acordo com o esperado. Esse processo fortaleceu a capacidade de análise e mostrou, na prática, a importância dos testes para a qualidade, confiabilidade e manutenção de um software. Além disso, também houve o incentivo constante por aprendizado. Durante o desenvolvimento, foi possível explorar novas tecnologias, conhecer diferentes formas de resolver problemas e descobrir áreas de interesse que antes não faziam parte da rotina. O PTA foi uma oportunidade de amadurecimento profissional, desenvolvendo não somente habilidades técnicas, mas também a capacidade de pensar de forma mais estratégica, crítica e colaborativa diante dos desafios encontrados ao longo do caminho.

Quanto ao gerente, Philip se mostrou comprometido com as entregas e com o desenvolvimento da equipe. No início, realizou um mapeamento das dificuldades e habilidades de cada integrante, com o objetivo de entender melhor o time e apoiar o crescimento de forma mais direcionada. Também incentivou o trabalho em equipe e a integração entre os membros. Ao longo do processo, valorizou as entregas realizadas, ao mesmo tempo em que trouxe feedbacks constantes e construtivos, contribuindo para a evolução técnica e comportamental do grupo.

Em relação à equipe, a colaboração foi um dos pontos mais fortes. O time trabalhou de forma bem integrada, com troca constante de conhecimentos e aprendizados entre os membros. Sempre que alguém tinha mais domínio em uma determinada área, esse conhecimento era compartilhado com os demais, o que ajudou a acelerar o desenvolvimento coletivo. Essa dinâmica criou um ambiente de apoio mútuo, onde dúvidas eram discutidas em conjunto e soluções eram construídas de forma colaborativa, fortalecendo tanto o resultado das entregas quanto o crescimento individual de cada integrante.

Como um todo, o PTA proporcionou não apenas uma oportunidade consistente de crescimento profissional, mas também um contato mais profundo com o Citi e com o Movimento Empresa Júnior. Ao vivenciar a rotina do programa no dia a dia, foi possível compreender melhor a importância do trabalho realizado, tanto em termos de desenvolvimento técnico quanto de formação pessoal e profissional. Houve também diversos momentos de integração entre os membros do CITi e os aspirantes, o que contribuiu bastante para a construção de um ambiente acolhedor e colaborativo. Em todos esses momentos, os membros mais experientes se mostraram sempre disponíveis para tirar dúvidas, orientar e auxiliar nos processos, o que facilitou muito o aprendizado e a adaptação ao longo da jornada. Essa proximidade ajudou a tornar a experiência mais completa, fortalecendo o senso de pertencimento ao time e permitindo um sentimento de pertencimento ainda durante o processo seletivo.

Nota final

A equipe responsável pelo projeto se compromete a manter esta documentação atualizada, clara e coerente com o estado atual da aplicação. Sempre que houver mudanças nas funcionalidades, tecnologias, estrutura do código ou fluxos de uso, o documento deverá ser revisado para refletir essas alterações. Essa atualização contínua garante que qualquer pessoa envolvida consiga compreender, utilizar, manter e contribuir com a solução de forma eficiente, mantendo a documentação como uma fonte confiável de consulta para o projeto.